

Day 1: Introduction to Artificial Intelligence(AI) and Machine Learning(ML)

Name: Manish Giri

Registration Number: 137256

Email: manishgiri8101@gmail.com

Assignment 1:

- i. Install Anaconda
- ii. Launch Jupyter Notebook
- iii. Run the first python cell
- iv. Submit a screenshot of output

IDE: VS Code, Kernal = base (Python 3.12.7) [~\anaconda3\python.exe]

Output:

```
day1_Introduction.ipynb X [Settings] [Run]
```

Day1 > day1_Introduction.ipynb > `arr = np.array([[1,2,3],[4,5,6]])`

Generate + Code + Markdown | Run All ...

[1] `print("Dated: 15-06-2026\nIntroduction to AI/ML")` 0.0s

... Dated: 15-06-2026
Introduction to AI/ML

[2] `import platform
print(platform.python_version())
print()` 0.0s

... 3.12.7

Assignment 2: Design a Library Management System using SSAD

Classroom Exercise — Design a Library Management System using SSAD					
Stage	Student Activity			Expected Output	
1. Problem Definition	Identify what the library system should do (issue books, track returns, manage users).			List of objectives and constraints	
2. Feasibility Study	Discuss technical and operational feasibility (hardware, software, user access).			Short feasibility report	
3. System Analysis	Draw a Data Flow Diagram (DFD) showing how data moves between modules.			Level-0 and Level-1 DFDs	
4. System Design	Create Entity-Relationship Diagram (ERD) for books, users, and transactions.			ERD with attributes and relationships	
5. Implementation & Testing	Write a simple Python prototype (e.g., book issue and return functions).			Working code snippet	
6. Maintenance	Suggest how to handle updates (new modules, bug fixes).			Maintenance plan summary	
 1.3K Participants  Chat  React  Share  AI Companion  More					

Answer:

1. Problem Definition:

The main goal is apply the concepts of Structured Systems Analysis and Design (SSAD) by designing a Library Management System. The system is supposed to automate day-to-day library operations like issuing books, tracking returns, managing members, and generating reports.

1.1. Objectives of the System:

- Allow users to search/request books from the library.
- Enable librarian to issue/receive books in a smooth and efficient way.
- Keep a proper record of all transactions including due dates and fines.
- Manage user registration and maintain their borrowing history.
- Generate transaction reports.
- Send find notifications to user who return books late.

1.2. Constraints:

- A member can borrow a maximum of 3 books at a time.
- The loan period is 14days, after that fine of Rs. 10/- per day will be charged.
- Only registered user with valid library card can borrow the book.
- Admin and librarians have separate access levels from regular members.

2. Feasibility Study:

Before starting the actual development, we need to determine whether the project is viable. Feasible study tells us whether the project makes sense from a technical, operational, and economic standpoint.

2.1. Technical Feasibility:

- System can be developed using existing technologies.
- Frontend: Python Tkinter (or HTML/CSS for web integration)
- Backend: Python
- Database: MySQL, SQLite
- Hardware Requirement: Standard computer with at least 4GB RAM.
- Development team has basic knowledge of Python and databases.

2.2. Operational Feasibility:

- Librarians and administrators require minimal training.
- User-Friendly interface with easy-to-understand.
- Also easy for non-tech stuff.
- Members can easy to search/borrow/track books.

2.3. Economic Feasibility:

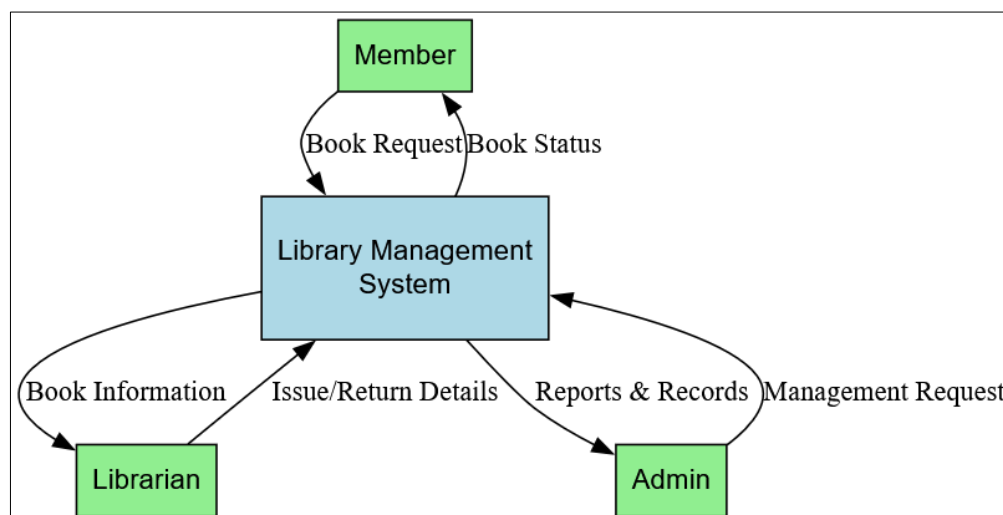
- Uses open-source tech such as python, MySQL/SQLite
- No software licensing costs.
- No skilled developer needed(Agile teams).

So, All three aspects are Feasible.

3. System Analysis:

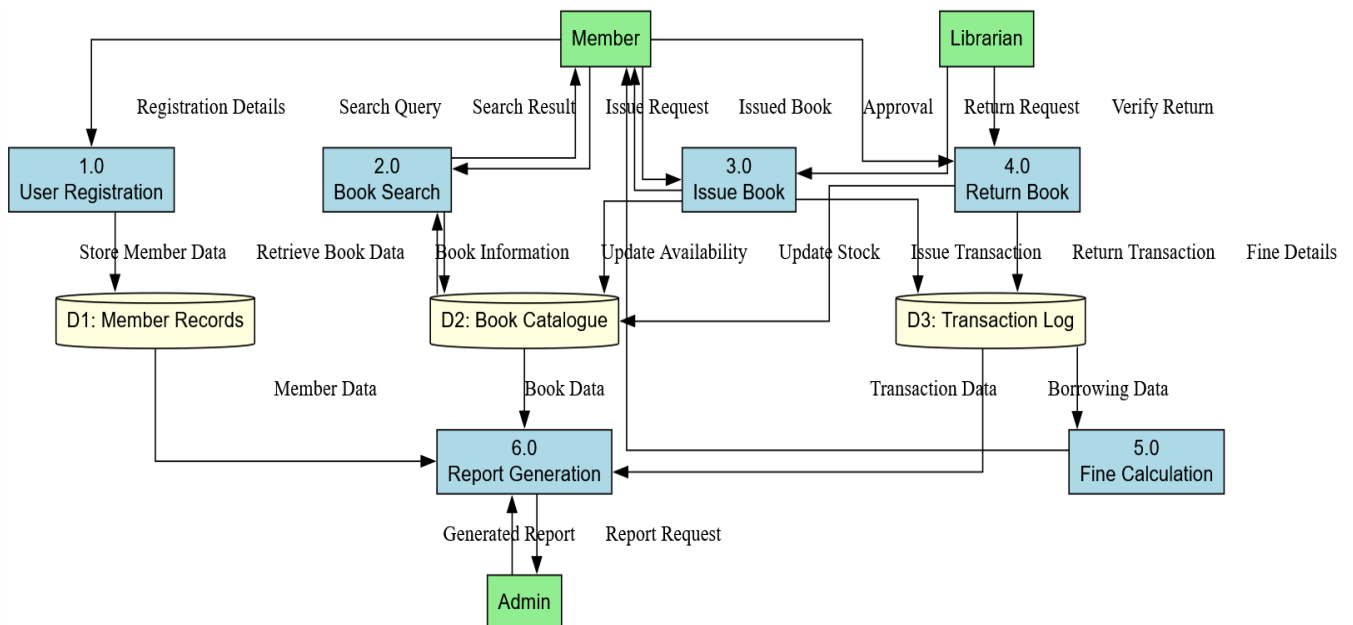
In this stage, we analyze how data moves throughout the system. We used Data Flow Diagrams (DFDs) to show this.

3.1. Context Diagram (Level-0 DFD) :



The context diagram of Library Management System shows the interactions between the library management system and other external entities such as Member, Librarian, and Admin. Member sends requests for books to the system and gets the status of the books as a response. Librarian gives information about issues and returns and gets information about books from the system. Admin sends management requests and gets the reports and records from the system. It shows the entire library management system as a single process.

3.2. Level 1 DFD:



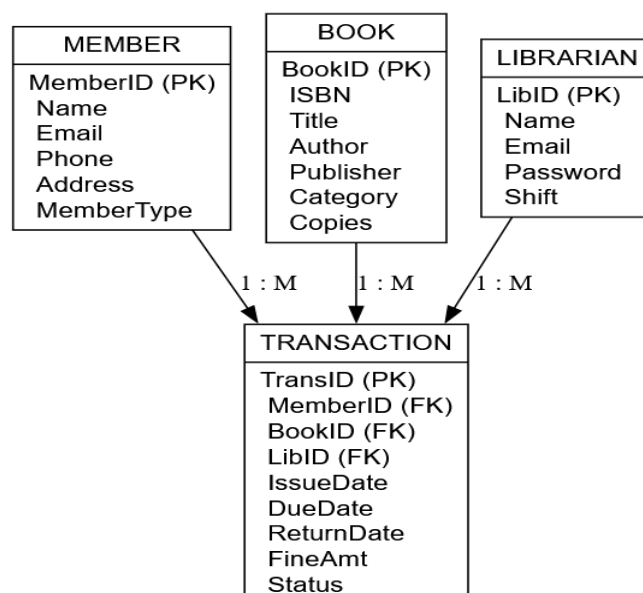
The Level-1 DFD shows six main processes:

- Process 1.0 – User Registration: Handles new member registration and stores data in Member Records.
- Process 2.0 – Book Search: Members can search books from the Book Catalogue.
- Process 3.0 – Issue Book: Librarian approves issue; transaction is logged in D3.
- Process 4.0 – Return Book: Member returns book; return date is updated.
- Process 5.0 – Fine Calculation: Checks due date and calculates fine if overdue.
- Process 6.0 – Report Generation: Admin gets summary reports from transaction log.

4. System Design:

System design involves creating the actual structure of the system. In this stage, we create the Entity-Relationship Diagram (ERD) which shows the entities (like Book, Member, Transaction) and the relationships between them, along with their attributes.

4.1. ERD:



The entity relationship diagram (ERD) of the library management system contains four main entities, namely: Member, Book, Librarian, and Transaction. The member borrows books through transactions, while the librarian controls the transactions. The transaction contains details such as the date of issue, date of expiry, date of return, fine charges, and transaction status. The relationship between them is one to many because one member, one book, and one librarian can conduct many transactions.

5. Implementation & Testing:

In this stage, we write a basic Python prototype to demonstrate two core functions of the Library Management System: issuing a book to a member and recording its return. This is just a simple working model to show that our design can be converted into actual code.

5.1. Python Code – Book issue & Return

```
# Using a Python dictionary to represent the database
books = {
    'B001': {'title': 'Let Us C', 'author': 'Yashavant Kanetkar', 'copies': 3},
    'B002': {'title': 'DBMS Concepts', 'author': 'Korth', 'copies': 2},
}
members = {
    'M001': {'name': 'Ravi Kumar', 'borrowed': []},
    'M002': {'name': 'Priya Sharma', 'borrowed': []},
}
transactions = []
#-----Book Issue-----
def issue_book(member_id, book_id):
    if member_id not in members:
        print('Member not found!')
        return
    if book_id not in books:
        print('Book not found!')
        return
    if books[book_id]['copies'] <= 0:
        print('Sorry, no copies available right now.')
        return
    if len(members[member_id]['borrowed']) >= 3:
        print('Borrow limit reached. Max 3 books allowed.')
        return
    books[book_id]['copies'] -= 1
    members[member_id]['borrowed'].append(book_id)
    transactions.append({'member': member_id, 'book': book_id,
                        'issue_date': '2025-06-15', 'due_date': '2025-06-29',
```

```

        'returned': False})
    print(f"Book '{books[book_id]['title']}' issued to
{members[member_id]['name']}")
#-----Book Return-----
def return_book(member_id, book_id):
    if book_id not in members[member_id]['borrowed']:
        print('This book was not issued to this member.')
        return
    books[book_id]['copies'] += 1
    members[member_id]['borrowed'].remove(book_id)
    for t in transactions:
        if t['member'] == member_id and t['book'] == book_id and not
t['returned']:
            t['returned'] = True
            t['return_date'] = '2025-07-02'
            break
    print(f"Book '{books[book_id]['title']}' returned by
{members[member_id]['name']}")

# ---- Test the functions ----
print('--- Issuing Book ---')
issue_book('M001', 'B001')
print()
print('--- Returning Book ---')
return_book('M001', 'B001')

```

5.2. Output:

```

--- Issuing Book ---
Book 'Let Us Python' issued to Ravi Kumar
--- Returning Book ---
Book 'Let Us Python' returned by Ravi Kumar

```

5.3. Testing:

TC	Input / Scenario	Expected Output	Result
TC1	Valid member issues a valid book	Book issued successfully	Pass
TC2	Member tries to issue unavailable book	'No copies available' message	Pass
TC3	Invalid member ID	'Member not found' message	Pass
TC4	Member already has 3 books	'Borrow limit reached' message	Pass
TC5	Valid return of issued book	Book returned, copies updated	Pass

6. Maintenance:

Once the system is deployed and being used in the library, it will need regular maintenance to keep it running properly and to add any new features that may be needed

in the future. Maintenance is an ongoing activity and is the final stage of the SSAD lifecycle.

Activity	Frequency	Responsible Person
Database Backup	Daily (automated)	System Admin
Bug Fixes	As and when required	Developer
Software Updates	Monthly	IT Team
New Module Addition	As per requirement	Developer + Librarian
User Training	When new features added	Librarian / IT Team
Performance Check	Every 3 months	System Admin

7. Feature Scope:

- Online web portal for members to search and reserve books from home.
- Email or SMS notification system to alert members about due dates and fines.
- Barcode/QR code scanning for faster book issue and return.
- Integration with university ERP system for student data synchronization.
- Mobile application for Android devices for better accessibility.

8. Conclusion:

In this way, we were able to gain an understanding of how to use the SSAD process in a practical situation. Having completed the six stages of the process from Problem Definition to Maintenance, we gained insights on how a software system undergoes analysis, design, and documentation prior to actual programming.

The Library Management System that we have designed satisfies all the requirements that a library of any college or school may need. It is evident from the DFDs that how the data flows within the system while ERD provides us an idea of what will be the structure of the database. We have created a python prototype that further proves the practicability of our design.

From this exercise, I learned how essential it is to plan and document the system prior to doing any coding.